

IFC Take-off — where every number comes from

QS Cost

July 30, 2026

CONTENTS

Reading the IFC Take-off panels: where every number comes from	
Why this document exists	
The chain in one picture	
Part 1 — Field guide	
The four tabs, and their headline hints	
1 · Selected element	
2 · Priced	
3 · Bill check	
4 · Measurable	
5 · Cost plan	
Part 2 — Technical appendix	
A · Which call feeds which panel	
B · One denominator, everywhere	
C · Measurement — <code>fe/model/ifcMeasure.ts</code>	
D · Pricing — <code>api/QsEarlyWarning.Core/Model/</code>	
The rate library	
The four declared rules	
The pricing loop — <code>TakeoffPricer.Price (TakeoffPricer.cs:94)</code>	
E · Bill check — <code>CompareToBoq (TakeoffPricer.cs:179)</code>	

F · The register — data/ifc_boq_map.csv

G · The EVM tiles

H · Zone placement and the two tiers

I · Period semantics

J · Provenance table

K · Sharp edges

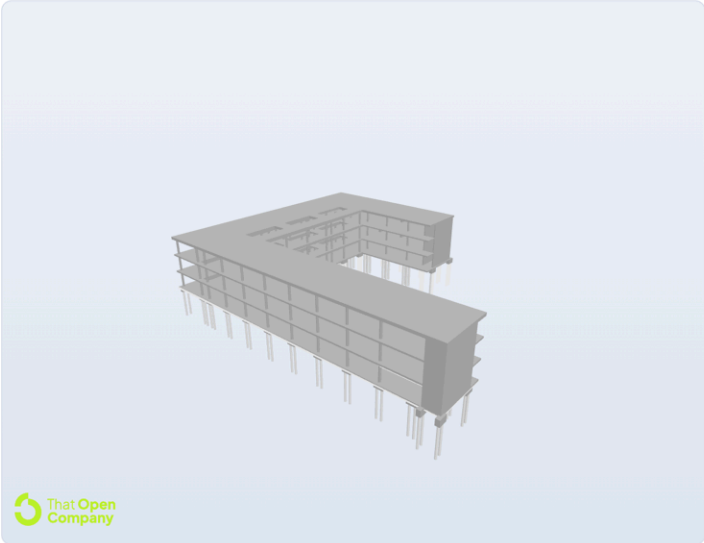
Honesty ledger

Reading the IFC Take-off panels: where every number comes from

Scope: the right-hand column of the **IFC Take-off** tab — the *Selected element* card and the four evidence tabs **Priced**, **Bill check**, **Measurable**, **Cost plan**. **Companions:** [22 — IFC Take-off](#) describes the feature; [17 — IFC → BOQ element register](#) describes the authored register. This document is the field-by-field derivation neither of them carries.

school_str.ifc

Load another IFC



That Open Company

This is not Tower X. It is a school's structural model (Autodesk Revit sample, IFC4) being priced with Tower X's rate library. The two buildings are unrelated — what is being demonstrated is that a rate library travels to

PRICED AT TOWER X'S RATES

4.6M AED

PRICEABLE SCOPE

4,638,842 AED

58%

MEASURABLE

883 of 1526 elements

✓ 508 priced + 375 unpriced + 643 unmeasured = 1526 elements in the model.

WHAT COULD BE PRICED

CLASS	QUANTITY	RATE	AMOUNT
IFCSLAB · 2.06	2,735.5 m³	1122.59	3,070,815 AED
IFCSLAB · 2.11	6,761.8 m²	181.80	1,229,315 AED
IFCWALL · 2.05	127 m³	1459.66	185,443 AED
IFCCOLUMN · 2.04	112.3 m³	1364.28	153,268 AED

WHAT COULD NOT — AND WHY

The IFC Take-off tab: model on the left, evidence panels on the right

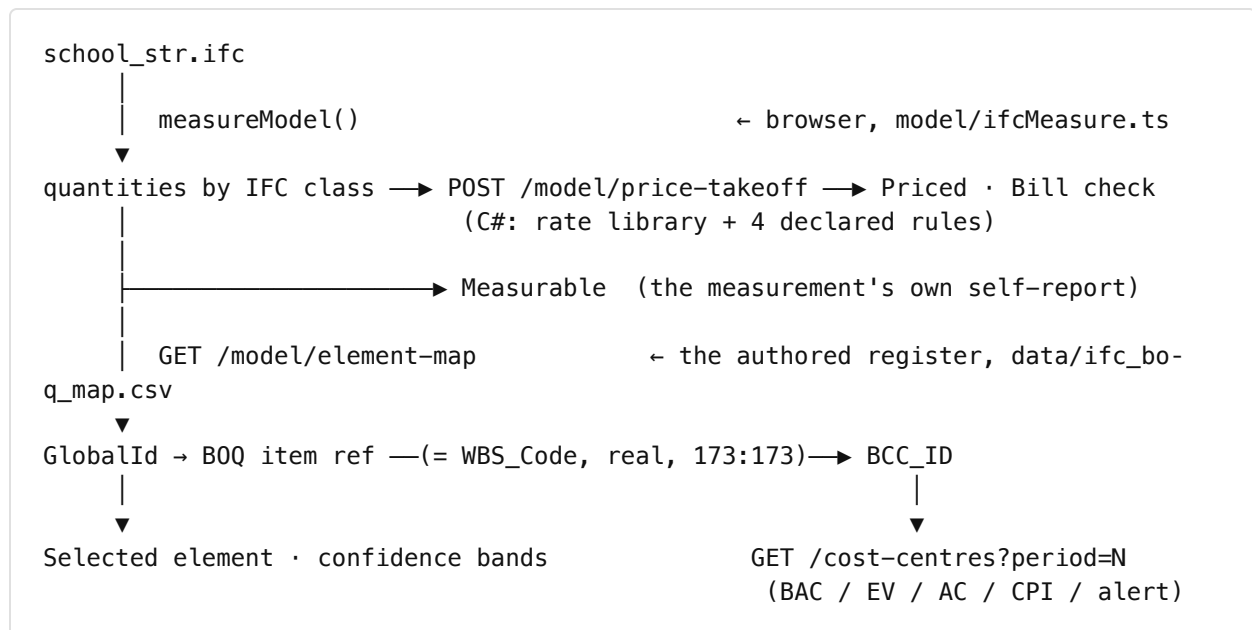
Why this document exists

Clicking an element in the model produces a column of numbers: a rate, a bill quantity, a CPI, a budget at completion, a confidence, a coverage percentage, a share placed. They look like one family of figures. They are not. They come from four different places, are computed by four different mechanisms, and carry four different levels of warranty:

Where a number comes from	How much to trust it
Measured off the loaded IFC	As good as what the exporter wrote — and this model carries no standard quantities
Priced by the project's rate library	Arithmetic on real BOQ rates; the <i>pairing</i> of class to item is a declared judgement
Read from the cost sheet via the register	Real project data, reached through one authored hop
Inferred by a class-and-storey rule	A statement about a category, not about that element

Nothing on the tab is invented, but the four are not interchangeable, and a QS acting on them needs to know which is which. That is what follows.

The chain in one picture



One arrow in that diagram is authored: **IFC element** → **BOQ item ref**. A genuine IFC export carries no cost codes, so that hop cannot be discovered and had to be declared, in a CSV a QS can read and argue with. Everything downstream of the BOQ item ref is source data.

Part 1 — Field guide

Each section says what the figure claims, how it is arrived at, and what would make it wrong.

The four tabs, and their headline hints

The tab labels carry their own numbers — `Priced · 375 unpriced`, `Measurable · 58%`, `Cost plan · 74% placed` — so switching tabs is a choice about what to read next, never the only way to discover that a number exists.

Tab	Hint shown on the label	What the hint is
Priced	<code>{n} unpriced</code>	<code>pricing.unpricedElements</code> — elements measured but with no rate
Bill check	<code>{n} lines</code>	<code>quantityVariances.length + uncomparableQuantities.length</code>
Measurable	<code>{n}%</code>	<code>measuredElements ÷ totalElements</code>
Cost plan	<code>{n}% placed</code>	<code>zoneMap.matchRate</code> — the class-and-storey placement rate

Note the two percentages are **not** the same denominator's story: 58% measurable is about whether the file carries a quantity; 74% placed is about whether a rule could locate the element in the cost plan. An element can be one without the other.

1 · Selected element

Appears only when the register resolves against the loaded model. Click any element; click it again to clear it.

Class / Storey / GlobalId. Read from the register row for that element, not re-derived from the model at click time. `Storey` is the IFC building storey that *contains* the element; an element in no storey reads `—`.

"In the bill" — one block per BOQ item the element consumes. A physical element consumes more than one bill item: a slab is concrete *and* its soffit formwork. Each block shows:

- **Item ref and description** — straight from `1_BOQ`.

- **Unit rate** — $\text{Direct+Indirect Amount} \div \text{Quantity}$ for that BOQ line. **Margin and contingency are excluded**: they are commercial positions taken per project, not transferable rates.
- **Bill quantity** — the quantity `1_BOQ` priced for that item. It is *Tower X's* quantity, shown for context. It is never used to price the loaded model.

A rate of 0.00 means “no rate”, not “free”. If the register points at a bill item the rate library has no priced line for, the item still lists — with a unit rate of zero. Read a zero rate as a gap in the bill, not as scope that costs nothing.

CPI and budget-at-completion tiles. These are the *cost centre's* numbers, not the element's. The element reaches a cost centre through its BOQ item ref, which is that centre's `WBS_Code`. The tile shows `CPI`, the centre's alert level, `BAC`, and `EV / AC` beneath it. There is no per-element cost anywhere in this system — the sheet records money per cost centre per period, and splitting it across 299 slabs would be a fabrication.

”Cost as at period N”. When the period slider is scrubbed past the last reported period, this line appears. The building keeps rising, because progress is forecast; the money does not move, because spend is not. Where the centre has a recent pace, the line also states the period it reaches 100% and the pace in percentage points per period. Where it has none, it says so rather than claiming a finish.

”Bound at confidence X”. `0.9` reads *declared by element class*; anything lower reads *inferred from the storey it sits on; the model carries no relationship to confirm it*. An element bound by several rules takes the **weakest** of them — a slab at 0.9 for concrete and 0.6 for something else reads 0.6.

”The bill prices nothing for this element.” Not a mapping failure. It is work the model contains and the estimate never carried — scope, and the earliest kind of gap a QS can act on.

Open {BCC-ID} hands the centre to the shared cost-centre drawer at the period the *data* is read at, never the projected one.

2 · Priced

The header card (always visible, above the tabs):

- **Priceable scope** — the money value of the part of the building that could be both measured and priced. On its own it is a misleading number, which is why it never appears without the residual beside it.
- **Measurable %** — $\text{measuredElements} \div \text{totalElements}$, with the raw counts beneath.

- **The tie-out line** — $\text{priced} + \text{unpriced} + \text{unmeasured} = \text{total elements}$. The total is the count the measurer reported **independently of the pricing pass**, so a pricing bug cannot hide behind a matching total. A X here means elements fell out unaccounted for and the priced figure is understating the building; the panel says so rather than showing a clean total.

What could be priced. One row per (IFC class, measure): quantity and unit, the BOQ item it prices through, that item's unit rate, and $\text{amount} = \text{quantity} \times \text{unit rate}$. Sorted by amount descending.

What could not — and why. The interesting half. Each row carries a reason in words a QS can act on — *this rate library prices no beam concrete, this library prices rebar by the tonne and converting bar geometry to tonnage needs a steel density and a bar schedule this model does not carry, no rate in this library for this element class.*

The unpriced residual is a **feature, not a gap to close**. Pointing beams at the slab rate so the picture fills in would attach cost to scope the estimate never carried.

Element counts on area rows read $-$ by design: area rides on elements already counted under their volume row, and counting them twice would break the tie-out.

3 · Bill check

What the model measures for a BOQ item, set against what that item was priced for.

Column	Meaning
Model vs bill	Measured quantity / the bill's quantity, in the item's unit
Variance	$\text{model} - \text{bill}$. Positive means the model carries more than was priced — the direction that costs money
(beneath)	$\text{variance} \div \text{bill quantity}$, rendered as a percentage
At this rate	$\text{variance} \times \text{unit rate}$ — the money the divergence is worth

Rows are grouped **by BOQ item, not by IFC class**: if two classes ever price through the same item, their quantities are two parts of one number, and comparing each class separately would report the same item as short twice. Sorted by absolute cost impact — the order a QS would work the list in.

Not compared. An item whose bill quantity is missing or zero is never compared; it is listed as uncomparable instead. Treating an absent quantity as zero would report a 100% overrun that exists only in the gap.

On the bundled demo this panel shows a mechanism, not an overrun. A school measured against Tower X's bill is two unrelated buildings. On a project's *own* model this is the earliest warning in the whole system: every other signal waits for cost to be booked, this one fires while the concrete is still a drawing.

4 • Measurable

Whether the file can be measured at all — the precondition for everything else on the tab.

Row	What it counts
Elements	Every element of the classes this take-off considers
Carrying a usable quantity	Those yielding a positive volume <i>or</i> area
In no building storey	Those the storey index could not place
Storeys	Distinct storey names found, listed beneath

The BaseQuantities warning. When it appears, the model carries no standard IFC BaseQuantities: a take-off written the textbook way returns nothing. The quantities shown were read from the exporter's own property sets, and the panel names the keys it used — which makes them exporter- and language-specific. The bundled Revit export is exactly this case: its numbers live in Spanish parameter groups.

Measured by class lists every class with its element count, volume and area. A means the class carries none of that measure — not that it measured zero.

5 • Cost plan

Whether the model can be *located* in a cost plan, and at what confidence.

- **% elements placed** — the share of elements a rule could put in a cost zone, with the counts beneath ().
- **Zones reached** — how many of the cost plan's zones got any geometry at all.

At what confidence. Which table appears depends on what resolved:

With the register (the bundled model): bands over the register's own confidences — **declared by element class** at 0.9, **inferred from storey** below that, **no bill item** at zero — each with its element count and share.

Without it (any other IFC you load): the two placement tiers —

Tier	Confidence	Meaning
Direct	0.90	The element's own properties literally name a zone code. It states where it belongs
Grouped	0.40	A class-and-storey rule placed it. True of the category, not of this element
None	—	No rule reached it

Grouped elements are drawn at reduced opacity — the visual form of that lower confidence.

These are two different scales and are never mixed. 0.9 / 0.6 is the register's *binding* confidence; 0.9 / 0.4 is the zone map's *placement* tier. The panel shows one or the other, never both.

"Nothing in this model links directly." Expected for a structural export, and worth knowing before trusting any model-driven cost figure: not one element carries a cost code in its property sets, so every placement is a rule's inference about a category.

The bindings (collapsed) is the rule-by-rule audit trail: class, BOQ item and role, element count, and the basis for each — reference material, not a finding.

In the model, not in the bill. Classes the register deliberately leaves unbound, with the reason. Not failures to map: scope the estimate never priced.

Zones with no geometry. Cost-plan zones this model contributed nothing to. A structural model carries no MEP, finishes or landscaping, and a match rate that ignored that would flatter itself.

Part 2 — Technical appendix

File paths are relative to the repository root; `fe/` abbreviates `QsEarlyWarning/frontend/qs-early-warning/src/`, `api/` abbreviates `QsEarlyWarning/src/`.

A · Which call feeds which panel

Panel	State	Source call
Selected element	<code>selected</code> , <code>centres</code> , <code>index</code>	<code>GET /api/v1/model/element-map</code> + <code>GET /api/v1/cost-centres?period=N</code>
Priced, Bill check	<code>pricing</code>	<code>POST /api/v1/model/price-takeoff</code>
Measurable	<code>measurement</code>	none — computed in the browser by <code>measureModel</code>
Cost plan	<code>zoneMap</code> , <code>links</code> , <code>index</code>	<code>GET /api/v1/model/cost-map?period=N</code> + the register

Orchestration is `fe/components/IfcTakeoff.tsx`. Two effects matter:

- `ingest` (`:122`) loads → measures → prices, once per file. It deliberately does *not* touch period-scoped data, because making it depend on period state would tear the viewer down and re-parse 8 MB of IFC on every scrub.
- **”locate in the cost plan”** (`:433`) refetches the cost map and cost centres whenever the read period changes, kept separate from painting so the right-hand figures follow the scrub whether or not the 4D sequence is the one drawing the model.

The register is fetched with **no period parameter** (`fe/api/client.ts:336`) — it is static, so scrubbing rejoins against the cost-centre array already in hand instead of refetching ~1,500 bindings.

B · One denominator, everywhere

`MEASURED_CLASSES` — 15 classes, `fe/model/ifcMeasure.ts:91` — fixes `totalElements`:

```
IFCSLAB, IFCCOLUMN, IFCBEAM, IFCWALL, IFCWALLSTANDARDCASE, IFCMEMBER, IFCPLATE,
IFCFOOTING, IFCPILE, IFCCOVERING, IFCCURTAINWALL, IFCREINFORCINGBAR, IFCSTAIR,
IFCRAMP, IFCROOF
```

Fetches with anchored regexes (`^IFCWALL$`) so `IFCWALL` cannot also sweep `IFCWALLSTANDARDCASE`. `ifcZoneMap` and `ifcCostLink` inherit that population, and `tools/ifc_boq_map/generate_map.py:51` mirrors the same list so the register's coverage is over a comparable denominator.

Every percentage on this tab is over those 15 classes, not over the whole IFC.

C · Measurement — `fe/model/ifcMeasure.ts`

Quantities are scraped from property sets, not from `IfcElementQuantity`. The bundled Revit 2024 → IFC4 export contains none; its numbers sit in Spanish parameter groups (`Dimensiones` → `Volumen`, `Area`). Two synonym tables carry the locales (`:19`, `:25`):

```
VOLUME_KEYS = ["volumen","volume","netvolume","grossvolume","net volume","gross
volume",
               "volumen neto","volumen bruto"]
AREA_KEYS    = ["area","área","netarea","grossarea","net area","gross area",
               "netsidearea","área neta","area neta"]
```

`pick()` (`:297`) returns the **first synonym present in list order** — there is no net-over-gross preference beyond that ordering. Property names are matched exactly after `trim().toLowerCase()`; no substring matching.

A quantity counts only if strictly positive (`:159`):

```
if (volume !== null && volume > 0) { measurement.volume += volume; measuremen-
t.volumeCount++; }
if (area !== null && area > 0) { measurement.area += area; measuremen-
t.areaCount++; }
if ((volume !== null && volume > 0) || (area !== null && area > 0))
measuredElements++;
```

So **measured = has volume OR area**, counted once per element; `volumeCount` and `areaCount` are independent tallies. Zero and negative values are treated as unmeasured.

There is no unit conversion anywhere in this chain. No `IfcUnitAssignment` is read, nothing is scaled, and the totals are summed exactly as authored — implicitly trusted to be m³ and m².

Storeys are resolved the spatial way, from `IfcBuildingStorey → ContainsElements`, not by walking up from the element (`:202`). A storey with no name reads `(unnamed)`; an element in no storey reads `(none)`. The whole index is wrapped in a `try/catch` — if it fails, every element reports unplaced. `unplacedElements` (`:175`) is then simply the measured ids the index has no entry for.

`baseQuantitiesEmpty` (`:189`) is true when none of the quantity keys actually seen is an IFC-standard base-quantity name (`netvolume`, `grossvolume`, `netarea`, `grossarea`, `net-sidearea`). A model carrying no quantities at all also reports true — the flag means “no standard base-quantity name was found”, i.e. the take-off rode on exporter parameter groups.

`byStorey` **counts are derived from the id lists** (`:130`), never accumulated separately, so a table count and the geometry it refers to cannot disagree.

D • Pricing — `api/QsEarlyWarning.Core/Model/`

THE RATE LIBRARY

`RateBook.From` (`RateBook.cs:58`) projects `1_B0Q` down to what pricing needs, and

```
private static double? UnitRateOf(BoqLine line)
{
    if (line.DirectIndirectAmount is { } amount && line.Quantity is { } qty &&
        qty > 0)
        return amount / qty;
    return null;
}
```

is the whole rate derivation: **`Direct+Indirect Amount ÷ Quantity`**. Lines without a positive rate are dropped — an item priced at zero cannot price anything, and letting it through would produce a confident `AED 0` rather than an honest “no rate”. `1_B0Q`’s margin and contingency columns are read into the domain model and never used here; that is the `rateBasis` claim made literal.

(The XML comment on `UnitRateOf` says the workbook carries a unit rate directly and division is a fallback. It does not — `BoqLine` has no unit-rate field and the code always divides. The behaviour is the division.)

THE FOUR DECLARED RULES

`TakeoffRateMap.Rules` (`TakeoffRateMap.cs:42`) — hard-coded on purpose, and shipped to the UI so it can be argued with:

IFC class	Measure	Unit	BOQ item	Rationale
IFCCOLUMN	volume	m ³	2.04	Column concrete, priced per m ³ of column
IFCWALL	volume	m ³	2.05	Structural wall concrete, priced per m ³ of wall
IFCSLAB	volume	m ³	2.06	Suspended slab concrete, priced per m ³ of slab
IFCSLAB	area	m ²	2.11	Slab soffit formwork, priced per m ² of slab face

`WhyUnpriced` (`:63`) supplies the residual's reasons for `IFCBEAM`, `IFCREINFORCINGBAR`, `IFCMEMBER` and `IFCPLATE`; anything else falls back to *"No rule maps {class} ({measure}) to a BOQ item."*

THE PRICING LOOP — `TAKEOFFPRICER.PRICE` (`TAKEOFFPRICER.CS:94`)

Per line, in order:

1. `unmeasuredElements += max(0, UnmeasuredCount)` — always, whatever else happens to the line.
2. `Quantity <= 0` → unpriced, *"No measurable quantity for this class in the model."*
3. No rule, or a rule pointing at an item the library has no rate for → unpriced, with the reason above.
4. **Unit guard:** `UnitsAgree(rule.Unit, rate.Unit)` after normalising `³→3`, `²→2`, `cum→m3`, `sqm→m2` and stripping spaces. Pricing m³ at an m² rate is silent nonsense, so it is refused.
5. Otherwise priced: `amount = round(quantity × unitRate, 2)`.

Then:

```
pricedAmount = round(Σ amount, 2)
accountedFor = pricedElements + unpricedElements + unmeasuredElements
tiesOut      = accountedFor == modelElementCount           // the measurer's inde-
pendent count
```

The client is what makes that tie-out meaningful (`IfcTakeoff.tsx:161–179`): a class contributes its element count to *either* `elementCount` or `unmeasuredCount` depending on whether any of its elements yielded a volume, and the **area line always carries** `elementCount: 0` so

the same elements are not counted twice. The API cannot enforce that invariant; it checks the sum.

E · Bill check — `CompareToBoq` (`TakeoffPricer.cs:179`)

Grouped by `BoqItemRef`, case-insensitively, over the **priced** lines only:

```
modelQuantity = Σ quantity of every priced line pointing at that item    (2 dp)
variance      = modelQuantity - boqQuantity                             (2 dp)
variancePct   = variance ÷ boqQuantity                                  ← a
fraction, unrounded
costImpact    = variance × unitRate                                     (2 dp)
```

A missing or non-positive `boqQuantity` short-circuits to `uncomparableQuantities` before any of that. Results are ordered by `|costImpact|` descending.

`variancePct` is the one unrounded numeric field in the payload, and it is a **fraction**: the UI multiplies by 100 at render (`IfcTakeoff.tsx:1063`).

F · The register — `data/ifc_boq_map.csv`

Authored by `tools/ifc_boq_map/generate_map.py`, which regex-parses the STEP file for four fields (GlobalId, entity class, storey containment, storey name). One row per (**GlobalId**, **BOQ item**), because an element consumes more than one bill item.

IFC class	Storey	BOQ items (role)	Confidence
IFCCOLUMN	any	2.04 concrete, 2.09 formwork	0.9
IFCWALL / IFCWALLSTANDARDCASE	any	2.05 concrete, 2.10 formwork	0.9
IFCSLAB	any	2.06 concrete, 2.11 formwork	0.9
IFCREINFORCINGBAR	Sub Level	2.12 rebar, raft	0.6
IFCREINFORCINGBAR	above ground	2.14 rebar, suspended slabs	0.6
anything else	—	none — written as an un- mapped row	0.0

```
DIRECT    = 0.9    # the class itself is the evidence
INFERRED  = 0.6    # the storey stands in for a relationship the file does not
carry
```

0.6 applies only to rebar, because `IfcRelAssignsToProduct` and `IfcRelNests` are both absent from the file: there is no way to know which column or slab a bar reinforces, and storey is the only signal left.

Loading (`api/QsEarlyWarning.Infrastructure/Excel/IfcElementMapCsvLoader.cs`) folds rows onto the GlobalId and takes the **minimum** confidence across an element's bindings — *"an element is only as well-placed as its shakiest binding"* (`:105`) — forcing 0 where an element ended with no refs. A hand-rolled RFC-4180 splitter is used because the `Basis` column carries commas inside quotes.

Serving (`api/QsEarlyWarning.Web.API/Controllers/ModelController.cs:116`) joins each item ref to the rate library for description / unit / rate / bill quantity, and to the cost centre through the panel:

```
// WBS_Code IS the B0Q item ref — an exact 1:1 in the source data.
var bccByItemRef = snapshot.Panel
    .Where(p => !string.IsNullOrEmpty(p.WbsCode))
    .GroupBy(p => p.WbsCode!.Trim(), StringComparer.OrdinalIgnoreCase)
    .ToDictionary(g => g.Key, g => g.First().BccId,
StringComparer.OrdinalIgnoreCase);
```

That second hop is not authored — it is a bijective join already present in the source data. One declared arrow buys the whole chain.

Resolving against the loaded model (`fe/model/ifcElementMap.ts:49`) is a single batched `getLocalIdsByGuids` call. Register rows the file does not contain are counted as `notInModel` and surfaced — *the register and this model have drifted apart* — never silently dropped. An element is “mapped” if it has at least one resolvable bill item, and `bandLabel` (`:43`) cuts the bands at `>= 0.9` → *Declared by element class*, `> 0` → *Inferred from storey*, else *No bill item*.

G · The EVM tiles

`GET /api/v1/cost-centres?period=N` (`DashboardController.cs:50`) filters the cached panel to that period and rounds. It computes nothing. The facts originate in `9_HISTORICAL_DATA` — `BAC_AED`, `PV_AED`, `EV_AED`, `AC_AED_Period` (cumulative in this workbook), `Plan_Pct_Complete`, `Actual_Pct_Complete`.

But the indices are recomputed, not read. On the database path the view `qs.cost_centre_evm` (`QsEarlyWarning/db/migrations/0003_evm_view.sql`) derives:

```

cpi          = round(EV,0) ÷ round(AC,0)          -- whole-AED operands, to match
the workbook exactly
spi          = round(EV,0) ÷ round(PV,0)
eac          = BAC × round(AC,0) ÷ round(EV,0)    -- CPI method; falls back to
BAC when nothing earned
vac          = BAC - EAC
alert_level  = AMBER iff CPI < 0.95, else GREEN    -- NOT_STARTED / CLOSED pass
through

```

The 0.95 threshold is `api/QsEarlyWarning.Domain/Constants/EvmThresholds.cs`. So a tile showing CPI and an alert is showing *the system's* EVM, computed from the sheet's cost facts — not the workbook's own `CPI` and `Alert_Level` columns.

`AC` resolves to the ledger's cumulative postings where a project has an active ledger, and to the sheet's `AC_AED_Period` otherwise.

H · Zone placement and the two tiers

`fe/model/ifcZoneMap.ts` declares 14 rules, first match wins in array order:

Zone	Classes
BASEMENT	IFCFOOTING, IFCPILE, IFCSLAB below ground
FLOORS-ALL	IFCSLAB at roof, IFCSLAB otherwise
STRUCTURE	IFCCOLUMN, IFCBEAM, IFCMEMBER, IFCREINFORCINGBAR, IFCWALL, IFCWALLSTANDARDCASE
EXTERNAL-FACADE	IFCCURTAINWALL, IFCPLATE
FLOORS-B2-RF	IFCCOVERING

The only heuristics are two storey predicates (`:24`):

```

const isBelowGround = (name: string) => /sub|basement|b[0-9]|below/i.test(name);
const isRoof        = (name: string) => /roof/i.test(name);

```

Matching runs **per (class, storey)**, not per class (`:92`), and an element in no storey can only match a rule that has no storey test. That is a fix, not a detail: an earlier version tested the storey condition against the whole class, so because the model contains a “Sub Level” the below-ground slab rule fired for *all* slabs and the tab reported a flattering 100% placed.

```
matchRate = matchedElements ÷ totalElements    // 0 when there are no elements
```

`IFCSTAIR`, `IFCRAMP` and `IFCR00F` are measured but have **no zone rule**, so they always land in `unmatched` and pull the match rate down — deliberately: a rule that guessed would inflate it. `zonesWithNoGeometry` reports the reverse direction, cost-plan zones this model reached not at all.

`fe/model/ifcCostLink.ts` grades placement:

```
export const TIER_CONFIDENCE: Record<LinkTier, number> = { Direct: 0.9, Grouped: 0.4 };
```

rendered as opacity `1` and `0.55`. Two passes, in this order: every element a zone rule placed is marked `Grouped` first, then a direct hit overwrites it — doing it the other way round would let a rule downgrade real evidence. Matching a property value to a zone code is **exact after normalisation** (`trim → upper → whitespace and underscores to hyphen`), never substring: `FLOORS-ALL` inside “concrete to floors, all levels” is prose, and substring matching would promote it to 0.9 evidence. Vocabulary codes shorter than 3 characters are dropped entirely — they match too much by accident to be trusted as identifiers.

`codeCarryingElements` is reported separately from `directCount` on purpose: it answers “was this model authored with cost in mind at all”, and **zero is the expected answer** for a structural export.

I · Period semantics

```
const dataPeriod = progress ? Math.min(viewPeriod, progress.originPeriod) : viewPeriod;
```

— `IfcTakeoff.tsx:391`.

The slider scrubs the model past the last reported period; every cost figure on the right holds at the origin. The workbook has no rows beyond it, so CPI, EV, AC and the drawer all resolve to the last measured period, and the panels label themselves as such rather than advancing. **This feature forecasts physical progress and nothing else** — deriving EV or AC from a projected percentage would manufacture a final-cost number with none of the validation such a number needs.

The projection itself (`GET /api/v1/forecast/progress`) is each centre's recent 3-period progress pace carried forward, with the error bar quoted on screen read off the shipped back-test rather than written into the caption by hand.

J · Provenance table

On screen	Derivation	Lives in
Class · Storey · GlobalId	Register row, resolved by GlobalId	<code>data/ifc_boq_map.csv</code>
Unit rate	$\frac{\text{Direct+Indirect Amount}}{\text{Quantity}}$ of the BOQ line	<code>RateBook.cs:84</code>
Bill quantity	<code>1_BOQ.Quantity</code>	<code>EstimateWorkbookReader.cs</code>
CPI · alert	$\text{EV} \div \text{AC}$ on whole-AED operands; AMBER below 0.95	<code>db/migrations/0003_evm_view.sql</code>
BAC · EV · AC	Sheet facts, rounded	<code>9_HISTORICAL_DATA</code>
Bound at confidence	Min over the element's authored bindings	<code>IfcElementMapCsvLoader.cs:105</code>
Priceable scope	$\sum (\text{quantity} \times \text{unit rate})$ over priced lines	<code>TakeoffPricer.cs:158</code>
Measurable %	$\frac{\text{measuredElements}}{\text{totalElements}}$	<code>ifcMeasure.ts:159</code> , <code>IfcTakeoff.tsx:513</code>
Tie-out line	$\frac{\text{priced} + \text{unpriced} + \text{un-measured}}{\text{totalElements}} ==$	<code>TakeoffPricer.cs:153</code>
Model vs bill, variance, at this rate	$\frac{\text{model} - \text{bill}}{\text{rate}}, \times \text{unit rate}$	<code>TakeoffPricer.cs:197</code>
Elements in no building storey	Measured ids absent from the storey index	<code>ifcMeasure.ts:175</code>
% elements placed	$\frac{\text{matchedElements}}{\text{totalElements}}$	<code>ifcZoneMap.ts:134</code>
Direct / Grouped confidence	<code>TIER_CONFIDENCE</code> constants	<code>ifcCostLink.ts:24</code>
Zones reached / with no geometry	Set difference against the cost map's zones	<code>ifcZoneMap.ts:137</code>

K · Sharp edges

- `variancePct` is a fraction, not a percentage — the only unrounded number in the variance payload.
 - `unitRate: 0` is indistinguishable from a free item on the element-map path (`ModelController.cs:141` falls back to zero rather than null when the rate library has no entry).
 - Area lines carry no element count by client-side convention; the tie-out depends on it and the API cannot enforce it.
 - Two rule tables share a name. `pricing.rulesApplied` is the 4 hard-coded pricing rules; `elementMap.rules` is the 8 authored bindings, which add column and wall formwork (2.09 / 2.10) and rebar (2.12 / 2.14) and carry confidence. They agree on 2.04 / 2.05 / 2.06 / 2.11 and diverge elsewhere — which is why the *Priced* tab and the *Cost plan* tab report different item sets for the same building.
 - Aggregation is worst-case, never averaged. An element's alert is its **worst** cost centre; its confidence is its **weakest** binding. A slab whose concrete is on budget and whose formwork is drifting is a slab with a problem, and averaging would hide it.
 - Coverage is capped by what the bill prices, not by the mechanism.
-

Honesty ledger

Claim on the tab	Status
Measured quantities	Read from the file's property sets — exporter- and language-specific, and unit-unchecked
Unit rates	Real , from <code>1_BOQ</code> , direct + indirect only
IFC class → BOQ item (pricing)	Declared judgement , 4 rules, shown on the page
IFC element → BOQ item (register)	Authored , in a CSV a QS can edit; join-integrity tests fail the build on a bad edit
BOQ item → cost centre	Source data — the item ref <i>is</i> the centre's <code>WBS_Code</code>
BAC / EV / AC	Source data , <code>9_HISTORICAL_DATA</code>
CPI / EAC / VAC / alert	Computed by the EVM view from those facts
Rebar → raft vs suspended slab	Inferred from storey at 0.6; the file carries no host relationship
Formwork rows	Inferred from the concrete element — a standard estimating implication, not a measurement
Zone placement (no register)	Inferred by class + storey at 0.4, unless the element names a zone itself
Which elements are built, and in what order	Assumed — the sheet records progress per cost centre, never per element
Cost past the last reported period	Never projected . Only progress is
The bundled model's AED total	A mechanism demonstration . A school priced with Tower X's library — the two buildings are unrelated, and the bill-check divergence is not an overrun

The one thing this system consistently refuses to do is invent a number. Where a figure cannot be derived from the data, the panel reports the gap instead — the unpriced residual, the uncomparable items, the unmapped classes, the centre with no pace. Those gaps are the output, not the noise around it.